PLP - Primer Parcial - 2do cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
				E	E	MB	P

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Entregar cada ejercicio en hojas separadas. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

El siguiente tipo de datos sirve para representar un sistema de archivos sin enlaces lógicos (lo llamamos FS por su sigla en inglés). Un sistema de archivos puede entenderse como un árbol de subdirectorios y archivos con la siguiente estructura:

```
data FS = Arch String | Dir String [FS] deriving (Eq, Show)
```

Donde `Arch nombre` representa el archivo con nombre `nombre`, y `Dir nombre contenido` representa el directorio llamado `nombre` que contiene los archivos y subdirectorios que aparecen en la lista `contenido`.

Definimos el siguiente directorio para los ejemplos:

```
unDir = Dir ``raíz'' [Arch ``arch1'', Arch ``arch2'', Arch ``arch3'',
          Dir ``subdir1'' [Arch ``arch4''], Dir ``subdir2'' [Arch ``arch1'', Arch ``arch5'']]
```

a) Dar el tipo y definir las funciones `foldFS` y `recFS`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo `FS`. Solo en este inciso se permite usar recursión explícita.
Ayuda: `recFS :: ... -> (String -> [FS] -> [a] -> a) -> ...`

b) Definir la función `rutas :: FS -> [String]`, que lista las rutas de todos los archivos y subdirectorios de un sistema de archivos dado, usando la barra "/" como separador de directorios.

Por ejemplo:

```
rutas unDir ⇝ [``raíz'',``raíz/arch1'',``raíz/arch2'',``raíz/arch3'',``raíz/subdir1'',
    ``raíz/subdir1/arch4'',``raíz/subdir2'',``raíz/subdir2/arch1'',``raíz/subdir2/arch5'']
```

c) Definir la función `válido :: FS -> Bool`, que indique si, para cada directorio de un sistema de archivos, todos sus archivos y subdirectorios tienen nombres distintos. No importa si hay nombres repetidos entre distintos niveles - por ejemplo, no hay problema si un directorio tiene el mismo nombre que uno de sus archivos o subdirectorios - o si dos subdirectorios contienen archivos con el mismo nombre; solo importan los nombres dentro de un mismo directorio.

Por ejemplo:

```
válido unDir ⇝ True
válido (Dir ``dir1'' [Arch ``arch1'', Arch ``arch1'']) ⇝ False
válido (Dir ``dir1'' [Dir ``dir2'' [Arch ``arch1'', Dir ``arch1'' []]]) ⇝ False
```

Sugerencia: usar `all` y `nub`.

d) Definir la función `rutasPosibles::[String]->[String]`, que dada una lista de nombres (sin repetidos) genera la lista (infinita) de todas las rutas no vacías que se pueden generar con esos nombres. Por ejemplo:

`rutasPosibles [``A'',``B'',``C'']` debe contener a ```A'',``B'',``C'',``A/A'',``C/B'',``A/C/C'',``B/A/A'',``B/A/B'',``C/A/B'',``A/A/B/B'',``C/B/C/A''` entre otras.

El código debe tener la siguiente estructura (a completar):

```
rutasPosibles xss = concatMap ... [0..]
    where
        ... = foldNat (\rs -> [s1++'/':s2 | ...]) ...
```

Con `foldNat::(b->b)->b->Integer->b` definido en la guía.

Ejercicio 2 - Demostración de propiedades

Considerar las siguientes definiciones[1]:

```
data Árbol23 a b = Hoja a | Dos b (Árbol23 a b) (Árbol23 a b)
                          | Tres b b (Árbol23 a b) (Árbol23 a b) (Árbol23 a b)
      const :: a -> b -> a
{C}   const = (\x -> \y -> x)
      tamaño :: Árbol23 a b -> Int
{A0}  tamaño (Hoja v) = 1
{A1}  tamaño (Dos x i d) = 1 + tamaño i + tamaño d
{A2}  tamaño (Tres x y i m d) = 2 + tamaño i + tamaño m + tamaño d
      truncar :: a -> Árbol23 a b -> Int -> Árbol23 a b
{T0}  truncar c (Hoja v) = const (Hoja c)
{T1}  truncar c (Dos x i d) =
        (\n->if n<1 then Hoja c else Dos x (truncar c i (n-1)) (truncar c d (n-1)))
{T2}  truncar c (Tres x y i m d) =
        (\n->if n<1 then Hoja c else Tres x y(truncar c i (n-1))(truncar c m (n-1))(truncar c d (n-1))
```

a) Demostrar la siguiente propiedad:

$$\forall\, \texttt{t::Árbol23 a b} \,.\, \forall\, \texttt{c::a} \,.\, \forall\, \texttt{n::Int} \,.\, \texttt{tamaño (truncar c t n)} \leq \texttt{tamaño t}$$

Es importante **plantear la propiedad como predicado unario** y **exhibir claramente el esquema de inducción estructural.**

Se consideran demostradas las propiedades conocidas sobre enteros y booleanos, así como también:

$$\{\texttt{LEMA}\}\ \forall \texttt{t::Árbol23 a b} . 1 \leq \texttt{tamaño t}$$

Para el caso del constructor `Tres` no es necesario escribir la demostración paso por paso, basta con enunciar la hipótesis inductiva y la tesis inductiva. Para los constructores `Hoja` y `Dos` sí es importante presentar la demostración completa.

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$((\pi \Rightarrow \rho) \vee (\sigma \Rightarrow \tau)) \Rightarrow (\sigma \wedge \pi) \Rightarrow (\rho \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda para modelar estructuras con forma de **anillos** finitos. Un anillo cuenta con un elemento actual, seguido por una cadena de 0 o más elementos hasta volver al actual.

Se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \cdots \mid \texttt{Anillo}_\tau \qquad M ::= \cdots \mid \langle M \rangle \mid M \cdot\!\!:\!\cdot M \mid \texttt{actual}(M) \mid \texttt{avanzar}(M)$ donde:

- $\texttt{Anillo}_\tau$ es el tipo de los anillos con elementos de tipo τ.
- $\langle M \rangle$ es un anillo cuyo único elemento (y por ende su elemento actual) es M.
- $M_1 \cdot\!\!:\!\cdot M_2$ es el anillo resultante de agregar el elemento M_2 a continuación del elemento actual de M_1. Notar que el elemento actual sigue siendo el mismo, y que este operador es un constructor (no realiza ninguna acción sobre el anillo ni sus elementos). Por ejemplo: $(\langle \underline{0} \rangle \cdot\!\!:\!\cdot \underline{2}) \cdot\!\!:\!\cdot \underline{1}$ es el anillo cuyo elemento actual es $\underline{0}$, seguido por $\underline{1}$, luego $\underline{2}$ para luego volver al $\underline{0}$.
- $\texttt{actual}(M)$ es el elemento actual del anillo M.
- $\texttt{avanzar}(M)$ es el anillo que resulta de avanzar una posición en el anillo M, de manera que su elemento actual es el último que se agregó en M (o el actual de M si tiene un único elemento).

Por ejemplo: $\texttt{avanzar}(\texttt{avanzar}((\langle \underline{0} \rangle \cdot\!\!:\!\cdot \underline{2}) \cdot\!\!:\!\cdot \underline{1})) \twoheadrightarrow (\langle \underline{2} \rangle \cdot\!\!:\!\cdot \underline{1}) \cdot\!\!:\!\cdot \underline{0}$

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que no es lo mismo avanzar un anillo con uno, dos o más elementos.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión: $\texttt{actual}(\texttt{avanzar}((\langle \underline{0} \rangle \cdot\!\!:\!\cdot \mathsf{Pred}(\underline{3})) \cdot\!\!:\!\cdot \underline{1}))$

d) Definir como macro la función $\texttt{siguiente}_\tau$, que dado un anillo con elementos de tipo τ devuelve el elemento siguiente al actual.

[1] Escritas con recursión explícita para facilitar las demostraciones.

1 a)

```
foldFS :: (String → sol) → (String → [sol] → sol) → FS → sol
foldFS fArch fDir fS = case fS of
  —— (Arch s) → fArch s
  —— (Dir s subFSs) → fDir s (map (foldFS fArch fDir) subFSs)
```

```
recFS :: (String → sol) → (String → [FS] → [sol] → sol) → FS → sol
recFS fArch fDir fS = case fS of
  —— (Arch s) → fArch s
  —— (Dir s subFSs) → fDir s subFSs (map (recFS fArch fDir) subFSs)
```

b)

```
rutas :: FS → [String]
rutas = foldFS (\s → [s]) (\s subs → s : (map (\ruta → s ++ '/' ++ ruta)
                                              (concat subs)))
```

c)

```
válido :: FS → Bool
válido = recFS (const True)
  —— (\s subFSs subsols → (all id subsols) &&
  —— length (map nombre subFSs) == length (nub (map nombre subFSs)))
```

Donde nombre :: FS → String es una auxiliar definida como:

```
nombre fS = case fS of
  —— (Arch s) → s
  —— (Dir s _) → s
```

d) ATRÁS

d] rotasPosibles xss = concatMap nombresConNBarras [0..]

— where nombresConNBarras = foldNat (\rs → [s1 ++ '/' : s2 | s1 ← rs, s2 ← xss]) xss

[String] → [String]

[String]

2 a) Para demostrar la propiedad, planteamos como predicado unario sobre t de tipo Árbol123 a b:

$\mathcal{P}(t) \triangleq \forall c :: a . \forall n :: Int . \text{tamaño (truncar } c\ t\ n) \leq \text{tamaño } t$

Ahora sí; para demostrar la propiedad, realizamos inducción estructural sobre la estructura de t, es decir:

- Probar $\forall x :: a . \mathcal{P}(\text{Hoja } x)$
- Probar $\forall x :: b . \forall i :: \text{Árbol123 } a\ b . \forall d :: \text{Árbol123 } a\ b .$
  $(\mathcal{P}(i) \wedge \mathcal{P}(d)) \Rightarrow \mathcal{P}(\text{Dos } x\ i\ d)$
- Probar $\forall x :: b . \forall y :: b . \forall i :: \text{Árbol123 } a\ b . \forall m :: \text{Árbol123 } a\ b . \forall d :: \text{Árbol123 } a\ b .$
  $\underbrace{(\mathcal{P}(i) \wedge \mathcal{P}(m) \wedge \mathcal{P}(d))}_{\text{H.I.}} \Rightarrow \underbrace{\mathcal{P}(\text{Tres } x\ y\ i\ m\ d)}_{\text{T.I.}}$

(A lo largo de la demostración, omito los cuantificadores durante la manipulación de ecuaciones, las variables fueron instanciadas universalmente ("sean x de tipo a, n de tipo Int, ...") adecuadamente)

- Caso Hoja x: desarrollo el lado izquierdo de $\mathcal{P}(\text{Hoja } x)$

$\text{tamaño (truncar } c\ (\text{Hoja } x)\ n) \overset{TO}{=} \text{tamaño (const (Hoja } c)\ n)$

$\overset{C}{=} \text{tamaño } ((\backslash x \to (\backslash y \to x))\ (\text{Hoja } c)\ n) \overset{B}{=} \text{tamaño } ((\backslash y \to (\text{Hoja } c))\ n)$

$\overset{B}{=} \text{tamaño (Hoja } c) \overset{AO}{=} 1$

Por otra parte, el lado derecho es tamaño (Hoja x), y como (Hoja x) :: Árbol123 a b podemos aplicar el {LEMA}: $1 \leq \text{tamaño (Hoja } x)$

Por tanto, como $1 = \text{tamaño (truncar } c\ (\text{Hoja } x)\ n)$, afirmamos que

$$\text{tamaño (truncar } c\ (\text{Hoja } x)\ n) \leq \text{tamaño (Hoja } x)$$

Es decir, vale $\mathcal{P}(\text{Hoja } x)$. ✓

• Caso Dos x i d : desarrollo el lado izquierdo de $\mathcal{P}$(Dos x i d)

tamaño (truncar c (Dos x i d) n)

$\overset{T1}{=}$ tamaño ((\n → if n < 1 then (Hoja c) else (Dos x (truncar c i (n-1)) (truncar c d (n-1)))) n)

$\overset{\beta}{=}$ tamaño (if n < 1 then (Hoja c) else (Dos x (truncar c i (n-1)) (truncar c d (n-1))))

Para analizar los casos del if, aplicamos el lema de generación sobre el booleano "n < 1":

■ (n < 1) = True:

tamaño (if True then (Hoja c) else (Dos x (truncar c i (n-1)) (truncar c d (n-1)))))

$\overset{\text{if-true}}{=}$ tamaño (Hoja c) $\overset{A0}{=}$ 1

Por el mismo argumento que en el caso base, vale que 1 ≤ tamaño (Dos x i d), por lo que tamaño el lado izquierdo es menor o igual al derecho, así que vale $\mathcal{P}$(Dos x i d)

■ (n < 1) = False:

tamaño (if False then (Hoja c) else (Dos x (truncar c i (n-1)) (truncar c d (n-1)))))

$\overset{\text{if-false}}{=}$ tamaño (Dos x (truncar c i (n-1)) (truncar c d (n-1)))

$\overset{A1}{=}$ 1 + (tamaño (truncar c i (n-1))) + (tamaño (truncar c d (n-1)))

Ahora, aplicamos nuestra hipótesis inductiva, se decir $\mathcal{P}$(i) y $\mathcal{P}$(d). En particular:

tamaño (truncar c i (n-1)) $\overset{HI}{\leq}$ tamaño i y tamaño (truncar c d (n-1)) $\overset{HI}{\leq}$ tamaño d

Por propiedad de Int, podemos aplicar las desigualdades en la suma que obtuvimos antes, así que:

1 + (tamaño (truncar c i (n-1))) + (tamaño (truncar c d (n-1)))

$\overset{\text{Int HI}}{\leq}$ 1 + tamaño i + tamaño d $\overset{A1}{=}$ tamaño (Dos x i d)

Entonces, aplicando la desigualdad y ~~segundo~~ operando sobre la ecuación, llegamos a que el lado izquierdo de $\mathcal{P}$(Dos x i d) es menor o igual al lado derecho. Por tanto, vale $\mathcal{P}$(Dos x i d) ✓

2] b) EN HOJA SIGUIENTE

2 b

Para demostrar el teorema, escribo una derivación de la fórmula en el contexto vacío $\emptyset$:

$$
\dfrac{
\dfrac{
\dfrac{

3 a) Introduzca las siguientes reglas de tipado:

$$\frac{\Gamma \vdash M : \tau}{\Gamma \vdash \langle M \rangle : \text{Anillo}_\tau}\ \text{T-Llave} \qquad \frac{\Gamma \vdash M : \text{Anillo}_\tau \quad \Gamma \vdash N : \tau}{\Gamma \vdash (M :: N) : \text{Anillo}_\tau}\ \text{T-Puntitos}$$

$$\frac{\Gamma \vdash M : \text{Anillo}_\tau}{\Gamma \vdash \text{actual}(M) : \tau}\ \text{T-Actual} \qquad \frac{\Gamma \vdash M : \text{Anillo}_\tau}{\Gamma \vdash \text{avanzar}(M) : \text{Anillo}_\tau}\ \text{T-Avanzar}$$

b) El conjunto de valores se extiende a $V ::= \dots \mid \langle V \rangle \mid V :: V$

Las reglas de reducción añadidas son: (reglas de congruencia) (reglas de cómputo)

$$\frac{M \to M'}{\langle M \rangle \to \langle M' \rangle}\ \text{E-Llave} \qquad \frac{M \to M'}{\text{actual}(M) \to \text{actual}(M')}\ \text{E-Actual} \qquad \frac{M \to M'}{\text{avanzar}(M) \to \text{avanzar}(M')}\ \text{E-Avanzar}$$

$$\frac{}{\text{actual}(\langle V \rangle) \to V}\ \text{E-ActualLlave} \qquad \frac{}{\text{actual}(W :: V) \to \text{actual}(W)}\ \text{E-ActualPuntitos}$$

$$\frac{}{\text{avanzar}(\langle V \rangle) \to \langle V \rangle}\ \text{E-Avanzar1} \qquad \frac{}{\text{avanzar}(\langle V \rangle :: W) \to \langle W \rangle :: V}\ \text{E-Avanzar2}$$

$$\frac{\text{Avanzar}((V_1 :: V_2) :: V_3) \to \text{avanzar}(V_1 :: V_3) :: V_2}{\text{avanzar}((V :: W) :: U) \to (\langle U \rangle :: \text{"sinLlaves"}(V)) :: W}\ \text{E-Avanzar3}$$

$$\frac{N \to N'}{M :: N \to M :: N'}\ \text{E-PuntitosR} \qquad \frac{M \to M'}{M :: V \to M' :: V}\ \text{E-PuntitosL}$$

No logré definir una regla de cómputo válida en E-Avanzar 3, ya que aunque me gustaría poder agregar todos los elementos del anillo V luego de $\langle U \rangle$, esto genera una reducción ~~válida~~ inválida dado que mantiene el uso de las llaves en el actual de V. inválido

Por ejemplo, en mi reducción: $\text{avanzar}((\langle 0 \rangle :: 2) :: 1) \to (\langle 1 \rangle :: \langle 0 \rangle) :: 2$

Si pudiera "eliminar" las llaves, llegaría al término esperado.

Papel de fibra de caña de azúcar.

c) $actual(avanzar((\langle \underline{0} \rangle :: Pred(\underline{3})) :: \underline{1}))$

$$\xrightarrow{\text{E-Actual, E-Avanzar, E-PuntitosL, E-PuntitosR, E-PredSucc}} actual(avanzar((\langle \underline{0} \rangle :: \underline{2}) :: \underline{1})) \longrightarrow\!\!\!\!\rightarrow (*)$$

(en ese orden)

No logré definir una regla de reducción para este caso, pero suponiendo que ya existe, en uno o más pasos debería reducir a lo siguiente:

$$(*) \longrightarrow\!\!\!\!\rightarrow actual((\langle \underline{1} \rangle :: \underline{0}) :: \underline{2}) \xrightarrow{\text{E-ActualPuntitos}} actual(\langle \underline{1} \rangle :: 0)$$

$$\xrightarrow{\text{E-ActualPuntitos}} actual(\langle \underline{1} \rangle) \xrightarrow{\text{E-ActualLlave}} \underline{1} \checkmark$$

d) $siguiente_\tau \equiv (\lambda a : Anillo_\tau \cdot actual(avanzar(a)))$